

Panorama de la programación de sistemas

Repaso de la Nota 01 y trabajo con el Cuadernillo 01

Manuel Soto Romero Araceli Liliana Reyes Cabello

Colegio de Ciencia y Tecnología, UACM

Semestre 2026-2

Pregunta para iniciar el repaso

¿qué software tiene que existir alrededor de un programa para que podamos prepararlo y ejecutarlo?

Punto de partida

Un programa no se ejecuta aislado: utiliza herramientas y servicios con responsabilidades distintas.

Software de sistemas y programación de sistemas

Software de sistemas

Conjunto de programas que apoya la operación y el uso de una computadora, y proporciona servicios para preparar, traducir, ejecutar o controlar otros programas.

Programación de sistemas

Área que se ocupa del diseño y la implementación de ese software y de los mecanismos que conectan programas, herramientas y recursos de cómputo.

El foco está en la infraestructura que permite usar y ejecutar programas.

Programación de aplicaciones y de sistemas

Programación de aplicaciones	Programación de sistemas
Se concentra en un problema de una persona u organización.	Se concentra en la infraestructura de software que permite preparar y ejecutar programas.
Su producto realiza una tarea concreta.	Su producto ofrece servicios a otros programas o administra recursos.
Pregunta: ¿qué resultado necesita la persona usuaria?	Pregunta: ¿qué mecanismos necesita el programa para ejecutarse?

La diferencia está en el foco principal, no en una jerarquía entre ambas.

Características de la programación de sistemas

apoya la operación
de otros programas

relaciona niveles
de abstracción

considera la
arquitectura
de la computadora

requiere interfaces
claras

atiende corrección
y manejo de errores

Recordemos

Una herramienta recibe una representación, realiza una tarea y produce otra representación o prepara la ejecución.

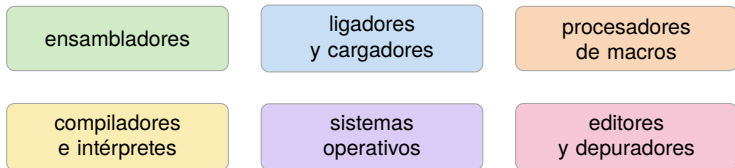
Trabajo con el cuadernillo

Cuadernillo de ejercicios 01

Ejercicios 1 y 2

- ▶ Ejercicio 1: completar las ideas centrales.
- ▶ Ejercicio 2: distinguir aplicaciones y software de sistemas.

Principales áreas y herramientas

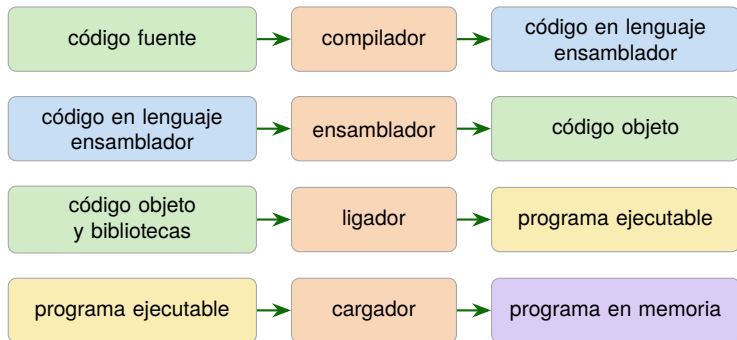


Las áreas se distinguen por su responsabilidad general y pueden colaborar en una misma cadena.

¿Qué responsabilidad tiene cada herramienta?

Herramienta	Responsabilidad general
Ensamblador	Traduce lenguaje ensamblador a código objeto o de máquina.
Ligador	Reúne código objeto y bibliotecas para formar un ejecutable.
Cargador	Coloca el programa en memoria y lo prepara para su ejecución.
Procesador de macros	Reconoce macros y las expande en secuencias del lenguaje.
Sistema operativo	Administra recursos y proporciona servicios para ejecutar programas.
Editor / depurador	Apoya la creación del programa / ayuda a observar su ejecución y localizar errores.

Del código fuente a la memoria



Es un recorrido panorámico: la forma concreta depende del sistema.

Lenguaje ensamblador y ensamblador

Lenguaje ensamblador

Es una **representación** del programa mediante instrucciones cercanas a la arquitectura de la computadora.

Ensamblador

Es la **herramienta** que traduce esa representación a código objeto o código de máquina.

representación $\neq$ herramienta que la procesa

Trabajo con el cuadernillo

Cuadernillo de ejercicios 01

Ejercicios 3 y 4

- ▶ Ejercicio 3: relacionar herramientas y responsabilidades.
- ▶ Ejercicio 4: completar el recorrido del código fuente a la memoria.

Compiladores e intérpretes

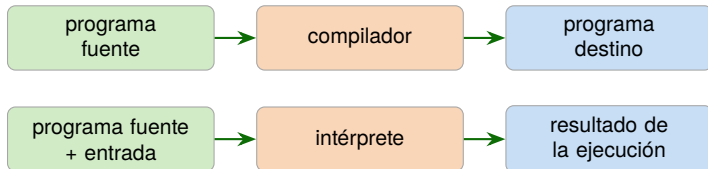
Compilador

Lee un programa en un lenguaje fuente y lo traduce a un programa equivalente en un lenguaje destino. También informa los errores que puede detectar durante la traducción.

Intérprete

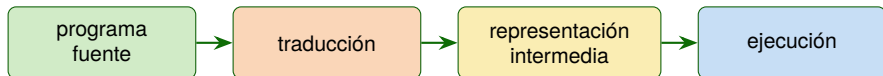
Ejecuta las operaciones especificadas por el programa fuente a partir de las entradas, sin producir como resultado principal un programa destino independiente.

Dos modelos de procesamiento



Traducir un programa no es lo mismo que ejecutar sus operaciones.

Los modelos pueden combinarse



Lo importante para el repaso

Los modelos básicos ayudan a distinguir responsabilidades. Un sistema real puede combinar traducción y ejecución sin que ambas acciones se vuelvan equivalentes.

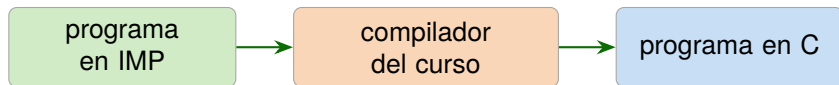
Trabajo con el cuadernillo

Cuadernillo de ejercicios 01

Ejercicios 5 y 6

- ▶ Ejercicio 5: comparar compilador e intérprete.
- ▶ Ejercicio 6: identificar el modelo descrito en cada situación.

El compilador del curso



¿Por qué es un caso integrador?

El compilador recibe programas, detecta errores, transforma representaciones y organiza componentes que deben colaborar mediante interfaces claras.

Cuatro ideas que recorrerán el curso

Especificar

definir las reglas

Transformar

cambiar la
representación

Dividir

separar
responsabilidades

Comprobar

verificar
resultados

Las cuatro aparecen en un mismo producto acumulativo, aunque se trabajarán progresivamente.

Trabajo con el cuadernillo

Cuadernillo de ejercicios 01

Ejercicios 7 y 8

- ▶ Ejercicio 7: relacionar decisiones con las cuatro ideas.
- ▶ Ejercicio 8: analizar el compilador de IMP a C.

Confusiones que debemos evitar

Confusión	Corrección
Compilador e intérprete hacen lo mismo.	Uno traduce; el otro ejecuta las operaciones indicadas.
El cargador forma el ejecutable.	El ligador forma el ejecutable; el cargador lo coloca en memoria.
Lenguaje ensamblador y ensamblador son lo mismo.	Uno es una representación; el otro es una herramienta.
La programación de sistemas se limita a sistemas operativos.	Incluye diversas herramientas que apoyan la operación y ejecución de programas.

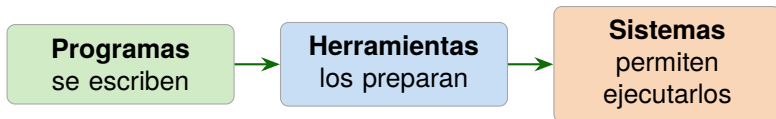
Trabajo con el cuadernillo

Cuadernillo de ejercicios 01

Ejercicio 9

- ▶ Detectar la parte incorrecta de cada afirmación.
- ▶ Reescribir cada idea de forma adecuada.

Conclusión



Idea de cierre

La programación de sistemas estudia y construye software que conecta lenguajes, programas y recursos de cómputo. El compilador será nuestro caso para observar esa relación de forma acumulativa.

Referencias

1. Soto Romero, M. y Reyes Cabello, A. L. (2026). *Programación de Sistemas. Nota de clase 01: Panorama de la programación de sistemas*. Colegio de Ciencia y Tecnología, UACM.
2. Soto Romero, M. y Reyes Cabello, A. L. (2026). *Programación de Sistemas. Cuadernillo de ejercicios 01: Panorama de la programación de sistemas*. Colegio de Ciencia y Tecnología, UACM.
3. Beck, L. L. (1996). *System Software: An Introduction to Systems Programming* (3.^a ed.). Addison-Wesley. ISBN: 978-0-201-42300-6.
4. Bala, K., Bracy, A., Sirer, E. y Weatherspoon, H. (2025). *Compilation—Assemblers, Linkers, & Loaders*. CS 3410 [diapositivas].
5. Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson/Addison-Wesley.
6. Vilar Torres, J. M. (2010). *Estructura de los compiladores e intérpretes*. Universitat Jaume I.